Università
di Genova

High Performance Computing

Report CUDA

Andrea Cattarinich 5137057

November 23, 2025

Contents

1 Architectures **2**

2 Hotspot Analysis **3**
2.1 Code Description 4

3 Vectorization **6**

4 Parallelization **7**

5 Conclusions **8**
5.1 Code description 8
5.2 Project Description 8
5.3 Sequential and Parallel Sections 8
5.4 Compilation 8
5.5 Hotspot Identification 8
5.6 Conclusions 8

1 Architectures

For this project, I used two workstations: my personal laptop and the Google Colab environment

HP Envy 17-ae103nl (Laptop)

CPU

- **Model**: Intel Core i7-6500U @ 2.50 GHz
- **Architecture**: x86_64
- **Physical cores**: 2
- **Logical threads**: 4 (Hyper-Threading)

GPU

- **Model**: NVIDIA GeForce GTX 950M
- **CUDA Compute Capability**: 5.0
- **Driver version**: 572.16
- **CUDA Version**: 12.8 (*CUDA Toolkit 12.x*)

Google Colab

CPU

- **Model**: Intel(R) Xeon(R) CPU @ 2.20GHz
- **Architecture**: x86_64
- **Physical cores**: 1
- **Logical threads**: 2 (Hyper-Threading)
- **Socket(s)**: 1

GPU

- **Model**: NVIDIA Tesla T4
- **CUDA Compute Capability**: 7.5
- **Driver version**: 550.54.15
- **CUDA Version**: 12.8 (*CUDA Toolkit 12.x*)

2 Hotspot Analysis

Heat conduction (or heat diffusion) is the process by which thermal energy spreads within a material due to temperature differences. In two dimensions, this phenomenon can be represented as heat spreading across a plate or surface over time. Numerical simulations of heat conduction are widely used in engineering, physics, and high-performance computing to predict temperature distribution and analyze thermal effects.

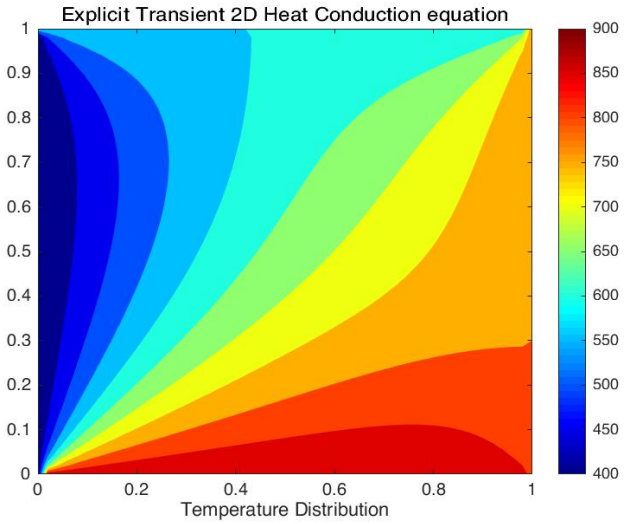

Figure 1: Illustration of 2D heat conduction simulation.

The formula to represent 2D heat conduction is given as follows:

$$\frac{\partial T}{\partial t} = \alpha \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right) \tag{1}$$

where:

- T is temperature;
- t is time;
- α is the thermal diffusivity;
- (x, y) are points in a grid.

To solve this problem numerically, one possible finite difference approximation is:

$$\frac{\Delta T}{\Delta t} = \frac{T_{i,j}^{n+1} - T_{i,j}^{n}}{\Delta t} = \alpha \left(\frac{T_{i+1,j}^{n} - 2T_{i,j}^{n} + T_{i-1,j}^{n}}{(\Delta x)^2} + \frac{T_{i,j+1}^{n} - 2T_{i,j}^{n} + T_{i,j-1}^{n}}{(\Delta y)^2} \right) \tag{2}$$

where:

- Δt is the timestep;
- i, j are indices in the grid;
- n indicates the current timestep, so $T_{i,j}^{n}$ is the temperature at point (i, j) at time $t = n\Delta t$, and $T_{i,j}^{n+1}$ is the temperature at the next timestep $t = (n+1)\Delta t$.

2.1 Code Description

This program simulates the diffusion of heat on a two-dimensional plate using the finite difference method. It includes two main implementations:

1. **Reference CPU version** - `step_kernel_ref()` - serial implementation
2. **Modified version** - `step_kernel_mod()` - intended for CUDA acceleration

Both implementations are designed to produce the same results.

Data Layout

The simulation uses a grid of size `ni × nj` (columns × rows).
Temperature values are stored in a **1D array**. A macro is used to convert 2D indices to linear index:

```
#define I2D(num, c, r) ((r)*(num)+(c))
```

Initialization

In the beginning, the boundary points of the grid are assigned with initial *random* temperatures, while the inner points update their temperature iteratively according to the formula above.

```
  for (int i = 0; i < ni * nj; ++i) {
      t1_ref[i] = t2_ref[i] =
      t1[i]     = t2[i]     =
          (float)rand() / (float)(RAND_MAX / 100.0f);
  }
```

TODO: aggiungere una fonte di calore concentrica al centro della griglia.

Execution

The core of the simulation is inside the `step_kernel()` function, which implements the approximation described in formula (2), shown below.

```
1  void step_kernel_mod(int ni, int nj, float fact, float* temp_in,
       float* temp_out)
2  {
3    int i00, im10, ip10, i0m1, i0p1;
4    float d2tdx2, d2tdy2;
5
6    // loop over all points in domain (except boundary)
7    for ( int j=1; j < nj-1; j++ ) {
8      for ( int i=1; i < ni-1; i++ ) {
9        // find indices into linear memory
10       // for central point and neighbours
11       i00 = I2D(ni, i, j);
12       im10 = I2D(ni, i-1, j);
13       ip10 = I2D(ni, i+1, j);
14       i0m1 = I2D(ni, i, j-1);
15       i0p1 = I2D(ni, i, j+1);
16
17       // evaluate derivatives
18       d2tdx2 = temp_in[im10]-2*temp_in[i00]+temp_in[ip10];
19       d2tdy2 = temp_in[i0m1]-2*temp_in[i00]+temp_in[i0p1];
20
21       // update temperatures
22       temp_out[i00] = temp_in[i00]+fact*(d2tdx2 + d2tdy2);
23     }
24   }
25 }
```

Note: The formula corresponds (2) to lines 18–23 in the above code.

Verification

After all timesteps, `temp1_ref` holds the CPU reference results, while `temp1` holds the modified version results. If the error exceed a threeshold (e.g. 0.0005), the simulation is considered correct.

```
1    float maxError = 0;
2
3    for( int i = 0; i < ni*nj; ++i ) {
4      if (abs(temp1[i]-temp1_ref[i]) > maxError) { maxError = abs(
         temp1[i]-temp1_ref[i]); }
5    }
6
7    // Check and see if our maxError is greater than an error bound
8    if (maxError > 0.0005f)
9      printf("The error is NOT within acceptable bounds.");
10   else
11     printf("The error is within acceptable bounds");
```

3 Vectorization

4 Parallelization

5 Conclusions

5.1 Code description

5.2 Project Description

5.3 Sequential and Parallel Sections

5.4 Compilation

5.5 Hotspot Identification

5.6 Conclusions